



Universidade do Minho

Unidade Curricular de Sistemas Operativos 2025/2026

Relatório de Projeto

**Bruno Freitas - A110016 , João Ribeiro - A111041 ,
Tiago Santos - A110313**

Data de Entrega: 10 de Maio de 2026

Índice

1	Introdução	1
2	Esquema Ilustrativo	2
3	Protocolo de Comunicação	2
3.1	Topologia das FIFOs	3
3.2	Estrutura Message	3
3.3	Tipos de mensagem	3
3.4	Fluxos de comunicação	3
3.5	Recolha dos filhos do -c	4
3.6	Vantagens da abordagem	4
4	Decisões de Implementação	4
5	Implementações e Funcionalidades	6
5.1	Operações do Cliente (runner)	6
5.2	Gestão do Servidor (controller)	6
6	Testes e Resultados	7
6.1	Análise das Políticas de Escalonamento (FCFS vs RR)	7
6.2	Impacto do Paralelismo	7
7	Conclusão	7

Índice de Figuras

1	Esquema ilustrativo do projeto	2
---	--	---

1 Introdução

No contexto da unidade curricular de Sistemas Operativos, foi proposto o desenvolvimento de um ambiente de orquestração de execução de comandos, com o objetivo de aplicar conhecimentos sobre processos, comunicação entre processos e manipulação de ficheiros no sistema operativo Linux.

O sistema é composto por dois componentes principais: um controlador (controller), responsável por gerir e escalonar os pedidos de execução recebidos, e um executor (runner), que permite ao utilizador submeter comandos e consultar o estado do sistema através da linha de comandos. A comunicação entre ambos é realizada exclusivamente através de pipes com nome (named pipes / FIFOs), recorrendo a chamadas ao sistema fornecidas pelo Linux.

Entre as funcionalidades implementadas, destacam-se a submissão e escalonamento de comandos, a consulta de comandos em execução e em espera, e o registo persistente da atividade do sistema. Para além destas funcionalidades base, o projeto inclui também suporte a operadores da linha de comandos, nomeadamente redirecionamento e pipes entre processos, execução paralela de múltiplos comandos com um grau de paralelismo configurável, e avaliação experimental de diferentes políticas de escalonamento.

Este relatório descreve a arquitetura do sistema, as decisões de implementação adotadas, os mecanismos de comunicação utilizados e os testes realizados, incluindo uma análise comparativa do impacto das políticas de escalonamento e das configurações de paralelismo na experiência dos utilizadores.

2 Esquema Ilustrativo

O esquema ilustrativo apresentado representa a arquitetura geral do sistema desenvolvido. No centro encontra-se o **controller**, o componente servidor que gere todas as comunicações e o escalonamento dos comandos. Cada **runner** (cliente) comunica com o **controller** através de dois canais distintos: um FIFO público partilhado por todos os runners, utilizado para o envio de pedidos, e um FIFO privado único por runner (identificado pelo seu PID), utilizado pelo **controller** para enviar respostas individuais. O fluxo de execução inicia-se com a submissão de um pedido pelo runner, seguida da autorização (ou enfileiramento) pelo controller, da execução do comando pelo runner, e por fim da notificação de conclusão enviada de volta ao controller.

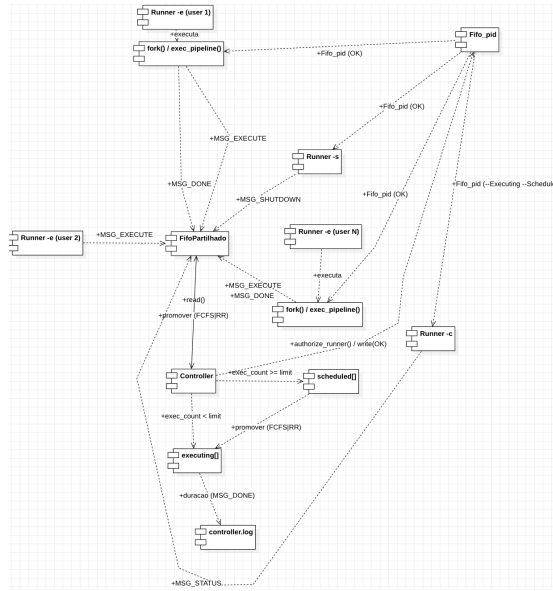


Fig. 1. Esquema ilustrativo do projeto

3 Protocolo de Comunicação

O sistema segue um modelo *cliente-servidor* baseado em *pipes* com nome (FIFOs). O **controller** é o servidor — único e persistente — e cada **runner** é um cliente efêmero que envia um pedido, aguarda a resposta e termina. A escolha de FIFOs justifica-se pela simplicidade, pela per-

sistência no sistema de ficheiros e por terem sido o mecanismo abordado em aula.

3.1 Topologia das FIFOs

O protocolo usa duas categorias de FIFO:

- **FIFO público** (`/tmp/controller_fifo`): criado pelo `controller` no arranque, é o ponto de entrada único do sistema. Todos os runners escrevem aqui, e o servidor lê continuamente. É aberto em `O_RDWR` para evitar que o descritor feche quando não há nenhum runner ligado.
- **FIFO privado por runner** (`/tmp/runner_<pid>_fifo`): criado pelo próprio runner, com base no seu PID. Serve para o servidor responder àquele runner em particular, evitando que respostas se cruzem entre clientes.

3.2 Estrutura Message

Toda a comunicação assenta numa `struct` de tamanho fixo definida em `protocol.h`, com os campos: `runner_pid`, `user_id`, `command_id`, `command[256]`, `msg_type` e `start_time`. O tamanho fixo permite que cada `read()` devolva exactamente uma mensagem, sem delimitadores nem parsing. Como `sizeof(Message)` é muito inferior a `PIPE_BUF` (4096 bytes em Linux), o POSIX garante que cada `write()` é atómico — mensagens de runners diferentes nunca se entrelaçam no FIFO público.

3.3 Tipos de mensagem

O campo `msg_type` distingue cinco operações:

- `MSG_EXECUTE` — submissão de um comando (`-e`).
- `MSG_DONE` — runner notifica o fim da execução.
- `MSG_STATUS` — pedido do estado das filas (`-c`).
- `MSG_SHUTDOWN` — pedido de encerramento (`-s`).
- `MSG_STATUS_DONE` — mensagem interna do servidor para recolha de zombies (ver 3.5).

3.4 Fluxos de comunicação

Submissão (-e). O runner cria o seu FIFO privado e abre-o em `O_RDWR` antes de contactar o servidor. Esta abertura prévia elimina uma *race*

condition: quando o controller mais tarde chamar `open(O_WRONLY)` para responder, já há um leitor registrado. O runner escreve então a `Message` no FIFO público; o controller autoriza imediatamente ("OK" no FIFO privado) ou enfileira o pedido. Após executar o comando — incluindo *pipelines* e redirecionamentos — o runner envia `MSG_DONE`, e o controller registra a duração no `controller.log` e promove o próximo da fila.

Consulta (-c). O controller faz `fork()` para o filho compor a resposta sem bloquear o loop principal. O filho escreve as listas `--Executing` e `--Scheduled` no FIFO privado do runner e termina (ver 3.5).

Encerramento (-s). O controller marca `shutdown_requested` e deixa de aceitar novos `MSG_EXECUTE`, mas continua a tratar os jobs em curso. Quando a fila de execução fica vazia, faz `unlink()` ao FIFO público e confirma ao runner que pediu o shutdown.

3.5 Recolha dos filhos do -c

Cada `fork()` para tratar um `-c` cria um processo que, ao chamar `exit()`, fica em estado *zombie* até ser recolhido. Em vez de `signal(SIGCHLD, SIG_IGN)` — fora do âmbito da disciplina — desenhámos um protocolo interno: antes de `exit()`, o filho envia `MSG_STATUS_DONE` ao pai pelo FIFO público com o seu PID. O loop principal, ao receber a mensagem, chama `waitpid()` com esse PID. Como o filho já fez `exit` antes da mensagem ser lida, o zombie já existe e o `waitpid` recolhe-o sem bloquear.

3.6 Vantagens da abordagem

- **Concorrência controlada** pelo limite `parallel-tasks`, sem deixar de aceitar novos pedidos.
- **Isolamento entre clientes** via FIFO privado nomeado pelo PID.
- **Mensagens atômicas** graças ao tamanho fixo abaixo de `PIPE_BUF`.
- **Recolha de zombies sem signal**, usando apenas FIFOs e `waitpid`.
- **Robustez a *race conditions*** pela abertura prévia em `O_RDWR` dos FIFOs.

4 Decisões de Implementação

Durante o desenvolvimento do projeto, foram tomadas várias decisões arquiteturais para garantir a simplicidade, robustez e eficiência do sistema:

- **Alocação estática das filas de espera e execução:** Optou-se por utilizar *arrays* estáticos (`executing[MAX_JOBS]` e `scheduled[MAX_JOBS]`) com um limite máximo generoso (1024), em detrimento de estruturas de dados dinâmicas, como listas ligadas. Esta abordagem simplifica a gestão de memória (eliminando chamadas a `malloc` e `free`) e anula a probabilidade de *memory leaks* e falhas de segmentação na gestão de múltiplos pedidos simultâneos.
- **Identificador único do comando (`command_id`):** O identificador único de comando, exigido pelo enunciado, é gerado pelo `runner` com base no seu próprio PID (`getpid()`). Como o sistema operativo garante a unicidade do PID de processos em execução, evitou-se a necessidade de sincronização complexa entre múltiplos clientes para atribuição de IDs.
- **Prevenção de *busy waiting* no servidor:** O `controller` abre o FIFO público com a flag `O_RDWR` (leitura e escrita), em vez de apenas `O_RDONLY`. Caso fosse aberto apenas para leitura, o `read()` retornaria EOF (0) sempre que o último `runner` terminasse, resultando num ciclo infinito de consumo de CPU. Ao manter uma extremidade de escrita aberta, a chamada bloqueia eficientemente até à chegada de um novo cliente.
- **Política Round-Robin baseada no Utilizador:** A política alternativa de escalonamento foi desenhada para garantir a justiça no *throughput*. Ao seleccionar o próximo processo da fila, o escalonador procura iterativamente o primeiro comando pertencente a um utilizador diferente daquele que foi escalonado na iteração anterior. Se a fila contiver apenas comandos de um único utilizador, a política degenera suavemente para FCFS, garantindo a ausência de inanição (*starvation*).

5 Implementações e Funcionalidades

5.1 Operações do Cliente (runner)

O programa **runner** é o ponto de interação com o utilizador e suporta três modos de operação estipulados: execução (**-e**), consulta (**-c**) e encerramento (**-s**).

No modo de execução, o principal desafio tecnológico reside no processamento de comandos complexos. A função `parse_command()` divide a *string* de entrada (fornecida pelo utilizador) em *tokens*, agrupa esses *tokens* em segmentos separados pelo operador de *pipe* (`|`), e deteta operadores de redirecionamento (`>`, `2>`, `<`), preparando matrizes de estruturas **Segment**.

Para a execução (`exec_pipeline()`), se o comando possuir N segmentos, o cliente cria $N - 1$ *pipes* anónimos. Em seguida, invoca `fork()` N vezes. Cada processo filho utiliza `dup2()` para:

- Ligar o **STDOUT** ao extremo de escrita do seu *pipe* e o **STDIN** ao extremo de leitura do *pipe* anterior.
- Sobrepor os descritores adequados caso o utilizador tenha especificado ficheiros de redirecionamento, os quais têm precedência sobre as ligações dos *pipes*.

Por fim, cada filho executa `execvp()`. O processo pai aguarda (`wait()`) que todos os componentes da *pipeline* terminem antes de notificar o servidor do sucesso da operação com a mensagem **MSG_DONE**.

5.2 Gestão do Servidor (controller)

O **controller** centraliza o orquestramento, garantindo que o número de processos a executar em simultâneo nunca ultrapassa o limite **parallel-tasks**.

O estado é mantido em duas filas: execução (**executing**) e espera (**scheduled**). O fluxo de gestão funciona por eventos:

- **Submissão:** Num pedido **MSG_EXECUTE**, se o limite de concorrência não tiver sido atingido, o servidor autoriza a execução via FIFO privado e insere o comando na fila **executing**. Se o sistema estiver lotado, insere o comando na fila **scheduled**.
- **Término e Registo:** Ao receber um **MSG_DONE**, o servidor calcula o tempo total de resposta (usando `gettimeofday()`) e regista-o persistentemente no ficheiro `controller.log`, juntamente com o **user_id** e o **command_id**. Depois de limpar a vaga na fila de execução, aplica a política de escalonamento escolhida (FCFS ou RR) para promover o próximo comando da fila **scheduled**, reativando o ciclo.

6 Testes e Resultados

6.1 Análise das Políticas de Escalonamento (FCFS vs RR)

No cenário de *fairness* (utilizador 1 com 4 jobs de 2s, utilizador 2 com 1 job de 1s), com paralelismo 1, o FCFS penaliza claramente o utilizador 2, que aguardou **8506 ms** de turnaround por estar no fim da fila. O RR reduziu esse valor para **4502 ms** (47% de melhoria), promovendo o job do utilizador minoritário assim que uma vaga ficou disponível. Com paralelismo 4, as duas políticas convergem (2500 ms), pois todos os jobs do utilizador 1 são admitidos em simultâneo e a distinção deixa de ser relevante. No cenário de *throughput* com carga distribuída por 3 utilizadores, FCFS e RR produzem resultados praticamente idênticos, dado que a fila já tem diversidade natural de utilizadores.

6.2 Impacto do Paralelismo

O nível de paralelismo revelou-se o fator dominante no desempenho global. No cenário de *throughput*, o tempo total desceu de **10033 ms** ($p = 1$) para **5074 ms** ($p = 2$) e **3071 ms** ($p = 4$), reduções de 50% e 69% respetivamente. O turnaround médio por job acompanha esta tendência, de 6382 ms para 1964 ms entre os extremos. O comportamento aproxima-se do modelo teórico T_{seq}/p , com um overhead residual de 50–70 ms atribuível à latência de comunicação via FIFO.

7 Conclusão

O sistema desenvolvido cumpre os objetivos propostos: o **controller** orquestra múltiplos **runners** via pipes com nome, garantindo escalonamento justo, registo persistente de execuções e suporte a paralelismo configurável. O **runner** suporta a execução de pipelines e redirecionamentos arbitrários, comunicando com o controller de forma assíncrona e sem condições de corrida.

A avaliação experimental demonstrou que o paralelismo é o fator com maior impacto no desempenho global, reduzindo o tempo total em cerca de 69% ao passar de $p = 1$ para $p = 4$. A política de escalonamento mostrou-se relevante sobretudo em cenários de baixo paralelismo com utilizadores dominantes: o RR reduziu o turnaround do utilizador minoritário em cerca de 47% face ao FCFS, ao promovê-lo antecipadamente na fila. Em cenários com carga equilibrada entre utilizadores, as duas políticas convergem para resultados equivalentes.

Como trabalho futuro, seria interessante explorar políticas mais sofisticadas, como escalonamento por prioridade dinâmica ou por quota de CPU por utilizador, bem como avaliar o comportamento do sistema sob cargas mais elevadas e com comandos de duração heterogénea mais acentuada.